

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**ARCHITECTURAL DESIGN SPECIFICATION
CSE 4316: SENIOR DESIGN I
FALL 2024**



**EXCALIBUR
LEARNLEAF ORGANIZER**

**EMMANUEL ASOMUGHA
IAN KELLY
ELLIOT MAI
BENJAMIN NICCUM
NICK PRATT**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	10.21.2024	EA, IK, EM, BN, IP	Document Creation
0.2	10.29.2024	EA, IK, EM, BN, IP	Complete Draft
1.0	11.04.2024	EA, IK, EM, BN, IP	Official Release

CONTENTS

1	Introduction	5
2	System Overview	6
2.1	Web Frontend Description	6
2.2	Web Backend Description	6
2.3	Mobile Frontend Description	7
2.4	Mobile Backend Description	7
2.5	Database Description	7
3	Subsystem Definitions & Data Flow	8
4	Web Frontend Subsystems	9
4.1	Pages	9
4.2	Navigation	10
4.3	Firebase Tools	11
5	Web Backend Subsystems	12
5.1	Pages	12
5.2	LearnLeaf Tools	13
5.3	Indexdb (Local Cache)	14
6	Mobile Frontend Subsystems	15
6.1	Pages	15
6.2	Navigation	16
6.3	Firebase Tools	16
7	Mobile Backend Subsystems	18
7.1	Pages	18
7.2	LearnLeaf Tools	19
8	Database Subsystems	20
8.1	User	20
8.2	Tasks	21
8.3	Projects	21
8.4	Subjects	21

LIST OF FIGURES

1	A simple architectural layer diagram	6
2	Data Flow	8
3	Web Frontend	9
4	Web application back end diagram	12
5	Mobile application front end diagram	15
6	Mobile application back end generalization	18
7	Mobile application back end diagram	18
8	Database Layer Structure	20

LIST OF TABLES

2	Page Interfaces	10
3	Navigation Interfaces	10
4	Firetools Interfaces	11
5	Pages Interfaces	13
6	LearnLeaf Tools Interfaces	14
7	IndexedDB Interfaces	14
8	Page Interfaces	16
9	Navigation Interfaces	16
10	Firebase tools Interfaces	17
11	Pages Interfaces	19
12	LearnLeaf tool Interfaces	19
13	Database User interfaces	20
14	Database Task interfaces	21
15	Database Project interfaces	21

1 INTRODUCTION

LearnLeaf Organizer is intended to be a one-stop shop organizational tool. It is primarily designed with students in mind, with many features being geared towards interacting with learning management systems and organizing tasks in a way that meets the format of higher education. The tool consists of two separate development trees that both interact with a shared database. Each development tree has both front and back-end components. This structure was chosen to limit the impact of development issues in one branch on the other. Reliability and availability are prioritized design features for the tool.

2 SYSTEM OVERVIEW

The LearnLeaf Organizer has a straightforward three-layer design consisting of a Frontend layer, a Backend layer, and a Database layer. Since the LearnLeaf Organizer will be available as both a website and a mobile app, the front end and back end needs to have two branches which are very similar but still distinct.

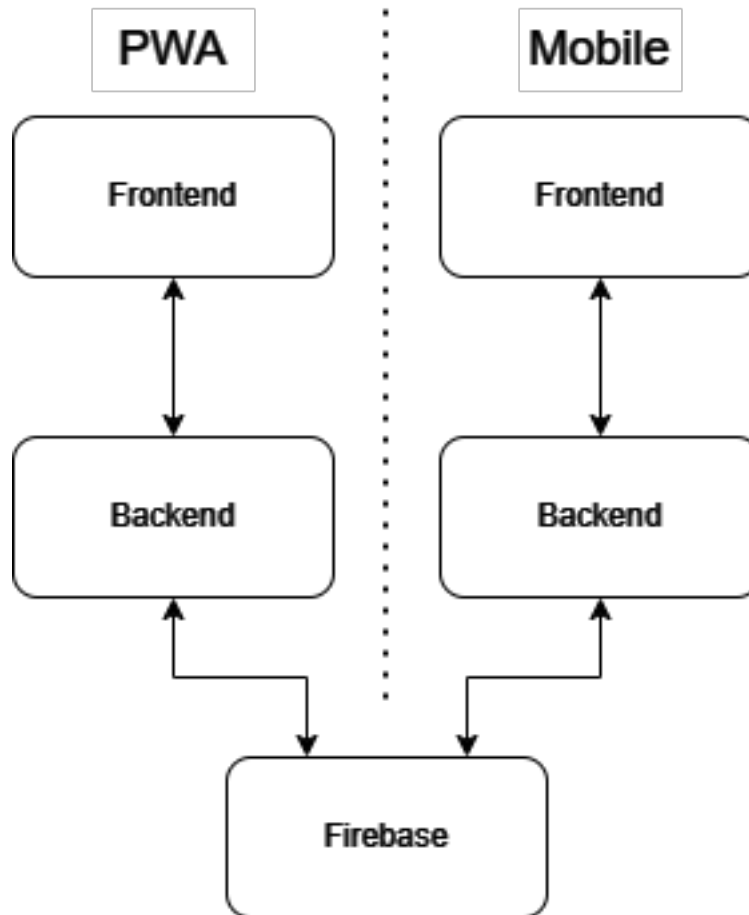


Figure 1: A simple architectural layer diagram

2.1 WEB FRONTEND DESCRIPTION

The Web Frontend consists of three main parts: Pages, Navigation, and Firebase Tools. The main function of the front-end is to present database information to the user in a usable form. It also acts as a way for the user to make modifications to and requests of the database which will be handled by the back-end.

2.2 WEB BACKEND DESCRIPTION

The Web backend consists of three main parts: Pages, Indexeddb (local cache) and the LearnLeaf tools. The purpose of the back-end is to facilitate communication and data transfer between the user and the database in a way that is consistent with database best practices, along with maintaining state consistency between the remote database and the local database cache.

2.3 MOBILE FRONTEND DESCRIPTION

The mobile frontend consists of 3 main parts: Pages, Navigation, and Firebase tools. The purpose of the mobile frontend is to present information from the database in an easy-to-read and useful format. It also acts as a way for the user to make modifications to and requests of the database which will be handled by the backend.

2.4 MOBILE BACKEND DESCRIPTION

The mobile backend is comprised of two main parts: Pages and the LearnLeaf Tools, which manage actions taken by the distinct pages in the Java code base, and act as an interface with Firebase's native database management tools. The purpose of the subsystem is to handle communication between the Firestore database and the user interface in a way that is consistent with database best practices.

2.5 DATABASE DESCRIPTION

The database is comprised of 4 main parts, user data, task data, project data, and subject data. The purpose of the database is to securely store each user's data, maintain data integrity, provide basic access control, maintain referential integrity, and respond to user queries.

3 SUBSYSTEM DEFINITIONS & DATA FLOW

The data flow within the mobile back-end subsystem is essential for ensuring seamless communication between the user interface, the LearnLeaf Utility Functions, and the Firestore database. This section outlines the various paths through which data travels, the transformations it undergoes, and the interactions between different components of the system. [1]

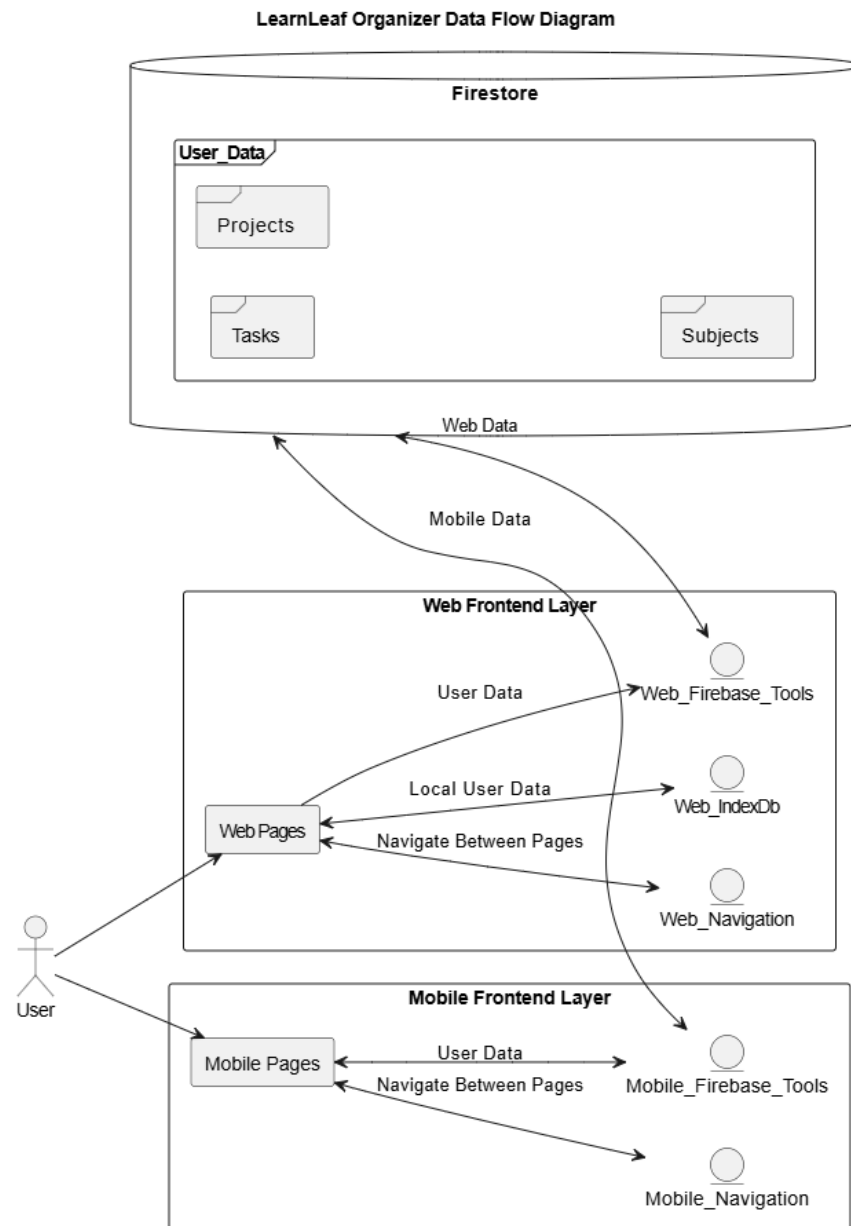


Figure 2: Data Flow

4 WEB FRONTEND SUBSYSTEMS

The Web Frontend layer consists of three main subsystems Pages, Navigation, and Firebase Tools.

4.1 PAGES

This subsystem represents the frontend web application, managing user interactions through various pages for task, subject, and project management, and account settings. See Figure 3.

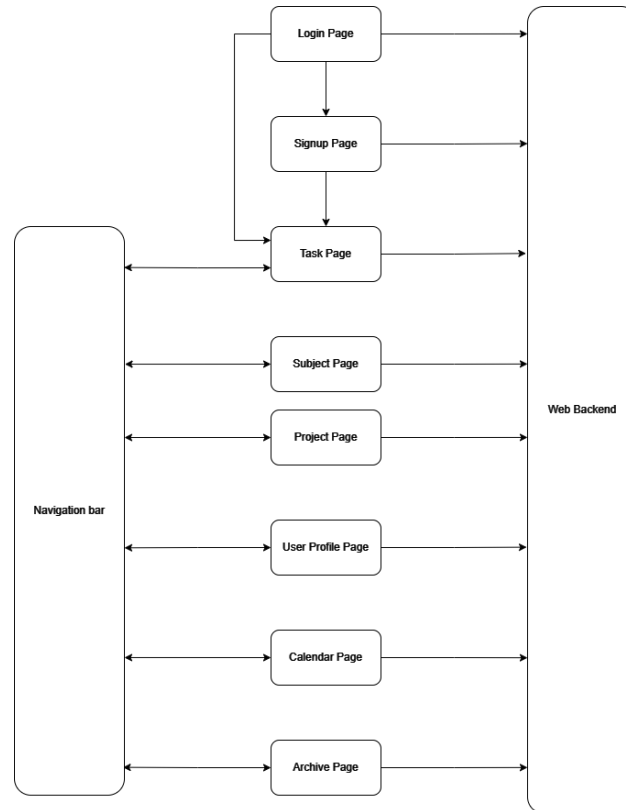


Figure 3: Web Frontend

4.1.1 ASSUMPTIONS

This subsystem assumes that Firebase tools are available and connected to handle backend data storage and authentication for the user interface pages. It also assumes a stable internet connection for accessing Firebase services.

4.1.2 RESPONSIBILITIES

The frontend web subsystem is responsible for providing a user interface that allows users to:

- Login and signup via the Login Page and Signup Page.
- Manage tasks on the Task Page by creating, editing, and deleting tasks.
- Manage subjects on the Subject Page by creating, editing, and deleting subjects.
- Manage projects on the Project Page by creating, editing, and deleting projects.
- Update user profile details, preferences, and account settings on the User Profile Page.

- View and manage calendar tasks on the Calendar Page.
- Access archived tasks, subjects, and projects on the Archive Page.

4.1.3 PAGE INTERFACES

Each of the subsystem interfaces and their respective incoming and outgoing data elements are defined in Table 8.

Table 2: Page Interfaces

ID	Description	Inputs	Outputs
1	Login	Username, Password	Success/Failure
2	Signup	Username, Password, Email	Success/Failure
3	Task	create/edit/delete	Task management status
4	Subject	add/Edit/Delete	management status
5	Project	add/edit/delete	management status
6	User Profile	add, preferences, account	Profile update status
7	Calendar	Task add/edit/delete	management status
8	Archive	view/edit/delete	management status

4.2 NAVIGATION

Sits at the top of every page and acts as an overarching connection.

4.2.1 ASSUMPTIONS

This subsystem assumed that page content would be loaded properly by each page respectfully.

4.2.2 RESPONSIBILITIES

The Navigation bar is responsible for allowing the user to jump between pages.

4.2.3 NAVIGATION INTERFACES

Each page will have a navigation bar overlay connecting it to every other page. See Table 9

Table 3: Navigation Interfaces

ID	Description	Inputs	Outputs
1	Task	Button Click	URL Redirect
1	Subject	Button Click	URL Redirect
1	Project	Button Click	URL Redirect
1	User Profile	Button Click	URL Redirect
1	Calendar	Button Click	URL Redirect
1	Archive	Button Click	URL Redirect

4.3 FIREBASE TOOLS

The Firetools subsystem is the main connection to the backend layer through which data is provided.

4.3.1 ASSUMPTIONS

This subsystem assumes that Firebase is functional properly.

4.3.2 RESPONSIBILITIES

The Firetools subsystem is responsible for fetching data when a page is loaded and syncing updates, edits, additions, and deletions with the backend and ultimately the database.

4.3.3 FIREBASE TOOL INTERFACES

Each page connects to the LearnLeafFunctions file which pulls in only function specific to that page. See Table 4

Table 4: Firetools Interfaces

ID	Description	Inputs	Outputs
1	Login	Login Request	User Info
2	Signup	Signup Request	Account Creation
3	Task	Task Request	Task List
4	Subject	Subject Request	Subject List
5	Project	Project Request	Project List
6	UserProfile	Request User Data	User Info
7	Calendar	Request Tasks	Task Data
8	Archive	Request Archives	Tasks, Subjects, Projects

5 WEB BACKEND SUBSYSTEMS

The web back-end is comprised of three main parts: Pages, Indexdb (local cache), and the LearnLeaf Tools, which manage actions taken by the distinct pages in the java code base, and act as an interface with Firebase's native database management tools. The purpose of the subsystem is to handle communication between the Firestore database, the local cached data, and the user interface in a way that is consistent with database best practices.

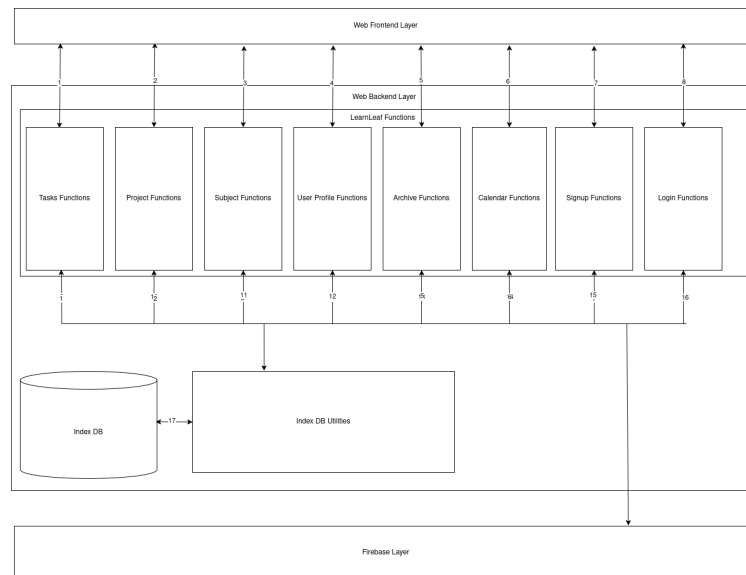


Figure 4: Web application back end diagram

5.1 PAGES

The pages subsystem consists of the communications between the individual pages and the LearnLeaf tools that process user input and pass it to the Firebase tools that manage the database.

5.1.1 ASSUMPTIONS

The primary assumption for the subsystem is that the user has access to the Firestore database. Because Firestore data is not stored initially on local memory, that means that an internet connection is required to initialize the local cache. Another assumption is that the LearnLeaf tools have access to Firebase native tools.

5.1.2 RESPONSIBILITIES

The pages subsection is responsible for:

- Transferring user input to the database interface
- Receiving query responses from the local database interface
- Passing user login requests to the remote database for validation

5.1.3 SUBSYSTEM INTERFACES

Table 5: Pages Interfaces

ID	Description	Inputs	Outputs
1	Create Entry	Entry ID Initial Information	
2	Edit Entry	Entry ID Edit Information	Process Outcome (Success/Failure)
3	Delete Entry	Entry ID	Process Outcome (Success/Failure)
4	Edit Preferences	Edit Information	Process Outcome (Success/Failure)
5	DB Query	User Input Query	
6	local DB Response		Result Entry(s)
7	Login Request	User Input Cre- dentials	Login Suc- cess/Failure

5.2 LEARNLEAF TOOLS

The LearnLeaf tools act as an interface between the pages and the Firebase database management tools.

5.2.1 ASSUMPTIONS

The primary assumption for the LearnLeaf Tools subsystem is that the user has access to the Firestore and locally cached database.

5.2.2 RESPONSIBILITIES

The Learnleaf Tools are responsible for:

- performing input validation
- processing user input data into database command entries
- passing queries from pages to the database
- passing query responses from database to pages
- updating the local database
- synchronizing the remote and local databases

5.2.3 SUBSYSTEM INTERFACES

Table 6: LearnLeaf Tools Interfaces

ID	Description	Inputs	Outputs
1	Local Database Update	Data Update Information	Local Data Changes for Remote
2	Sync Request	Current Remote State	Local Updates
3	DB Query to local	Entry ID	Entry ID
4	Remote Update	Local Data Changes	Success/Failure

5.3 INDEXDB (LOCAL CACHE)

The Indexdb/local cache is a repository for the users' data that increases response speed. It is essentially a locally stored copy of the users' database information.

5.3.1 ASSUMPTIONS

The assumptions made by the local cache subsystem are that the system running LearnLeaf has enough memory available to store the local copy of the DB and that there is an internet connection that will allow the remote database to synchronize with the local cache.

5.3.2 RESPONSIBILITIES

The IndexDB is responsible for:

- storing a copy of the users' data in local memory
- keeping itself up to date with the remote database
- responding to user queries
- reading updates from the LearnLeaf tools and modifying its contents
- pushing updates to the LearnLeaf tools to be passed to the remote database

5.3.3 SUBSYSTEM INTERFACES

Table 7: IndexDB Interfaces

ID	Description	Inputs	Outputs
1	Modify Entries	Entry ID Initial Information	Process Status(Success/Failure)
2	Sync with Remote	Entry Update Data	Process Status(Success/Failure)
3	Push to Remote		Entry Update Data
4	Query Response	Entry ID	Database Entry Data

6 MOBILE FRONTEND SUBSYSTEMS

The mobile frontend consists of 3 main parts: Pages, Navigation, and Firebase Tools. The purpose of the mobile front-end is to present information from the database in an easy-to-read and useful format. It also acts as way for the user to make modifications to and requests of the database which will be handled by the back-end.

6.1 PAGES

This subsystem represents the most visible part of the frontend web application, displaying database information, managing user interactions through various pages for task, subject, and project management, and account settings.

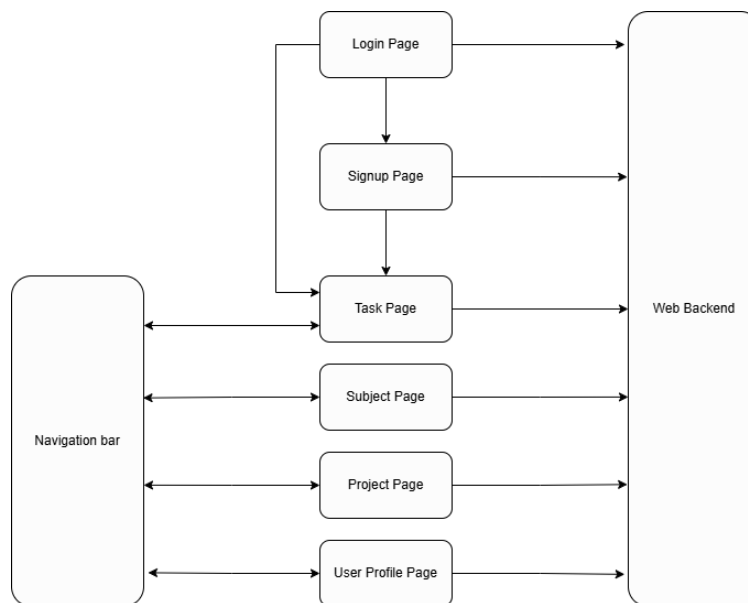


Figure 5: Mobile application front end diagram

6.1.1 ASSUMPTIONS

This subsystem assumes that Firebase tools are available and connected to handle back-end data storage and user authentication. It also assumes a stable internet connection for accessing Firebase services.

6.1.2 RESPONSIBILITIES

The web frontend subsystem is responsible for providing a user interface that allows users to:

- Login and signup via the Login Page and Signup Page.
- Manage tasks on the Task Page by creating, editing, and deleting tasks.
- Manage subjects on the Subject Page by creating, editing, and deleting subjects.
- Manage projects on the Project Page by creating, editing, and deleting projects.
- Update user profile details, preferences, and account settings on the User Profile Page.
- View and manage calendar tasks on the Calendar Page.
- Access archived tasks, subjects, and projects on the Archive Page.

- Display all query responses in a neat format

6.1.3 SUBSYSTEM INTERFACES

Table 8: Page Interfaces

ID	Description	Inputs	Outputs
1	Login	Username, Password	Success/Failure
2	Signup	Username, Password, Email	Success/Failure
3	Task	create/edit/delete	Task management status
4	Subject	add/Edit/Delete	management status
5	Project	add/edit/delete	management status
6	User Profile	add, preferences, account	Profile update status
7	Calendar	Task add/edit/delete	management status
8	Archive	view/edit/delete	management status

6.2 NAVIGATION

The Navigation subsystem sits at the top of every page and acts as an overarching connection to navigate between individual HTML pages.

6.2.1 ASSUMPTIONS

This subsystem assumes that each page exists and has valid code.

6.2.2 RESPONSIBILITIES

The Navigation bar is responsible for allowing the user to move between pages.

6.2.3 NAVIGATION INTERFACES

Each page will have a navigation bar overlay connecting it to every other page.

Table 9: Navigation Interfaces

ID	Description	Inputs	Outputs
1	Task	Button Click	URL Redirect
1	Subject	Button Click	URL Redirect
1	Project	Button Click	URL Redirect
1	User Profile	Button Click	URL Redirect
1	Calendar	Button Click	URL Redirect
1	Archive	Button Click	URL Redirect

6.3 FIREBASE TOOLS

The Firebase tools subsystem is the main connection to the backend layer through which data is provided.

6.3.1 ASSUMPTIONS

This subsystem assumes that it has access to the database through an internet connection or local storage.

6.3.2 RESPONSIBILITIES

The Firetools subsystem is responsible for:

- Fetching data when a page is loaded
- Syncing updates between local and remote databases
- Passing edits, additions, and deletions to both local and remote databases
- Passing queries to the local database

6.3.3 FIREBASE TOOL INTERFACES

Each page connects to the LearnLeafFunctions file which pulls in only Firebase functions specific to that page.

Table 10: Firebase tools Interfaces

ID	Description	Inputs	Outputs
1	Login	Login Request	User Info
2	Signup	Signup Request	Account Creation
3	Task	Task Request	Task List
4	Subject	Subject Request	Subject List
5	Project	Project Request	Project List
6	UserProfile	Request User Data	User Info
7	Calendar	Request Tasks	Task Data
8	Archive	Request Archives	Tasks, Subjects, Projects

7 MOBILE BACKEND SUBSYSTEMS

The mobile back-end is comprised of two main parts: Pages and the LearnLeaf tools, which manage actions taken by the distinct pages in the Java code base, and act as an interface with Firebase's native database management tools. The purpose of the subsystem is to handle communication between the Firestore database and the user interface in a way that is consistent with database best practices.

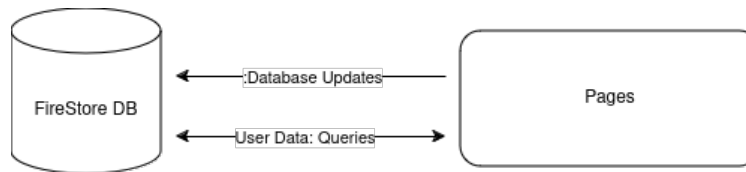


Figure 6: Mobile application back end generalization

7.1 PAGES

The pages subsystem consists of the communications between the individual pages and the LearnLeaf tools that process user input and pass it to the Firebase tools that manage the database.

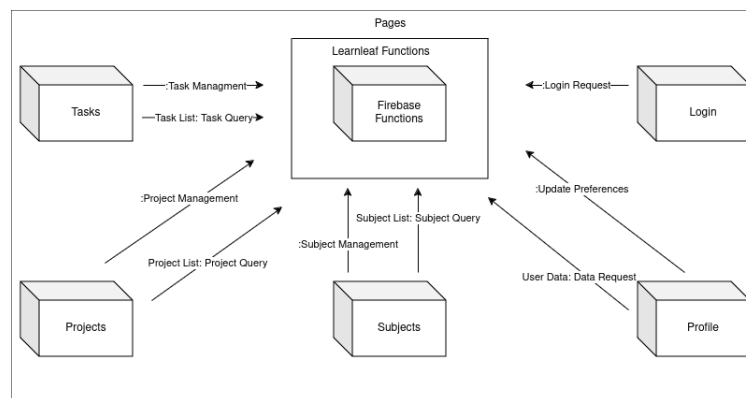


Figure 7: Mobile application back end diagram

7.1.1 ASSUMPTIONS

The primary assumption for the subsystem is that the user has access to the Firestore database. Because Firestore is not stored locally on mobile, that means that an internet connection is required. Another assumption is that the LearnLeaf functions have access to Firebase native tools.

7.1.2 RESPONSIBILITIES

The pages subsection is responsible for:

- Transferring user input to the database interface
- Receiving query responses from the database interface
- Passing user login requests to the database for validation

7.1.3 SUBSYSTEM INTERFACES

Table 11: Pages Interfaces

ID	Description	Inputs	Outputs
1	Create Entry	Entry ID Initial Information	NA
2	Edit Entry	Entry ID Edit Information	Process Outcome (Success/Failure)
3	Delete Entry	Entry ID	Process Outcome (Success/Failure)
4	Edit Preferences	Edit Information	Process Outcome (Success/Failure)
5	DB Query	User Input Query	Result Entry(s)
6	Login Request	User Input Cre- dentials	Login Suc- cess/Failure

7.2 LEARNLEAF TOOLS

The LearnLeaf tools act as an interface between the pages and the Firebase database management tools.

7.2.1 ASSUMPTIONS

The primary assumption for the LearnLeaf tools subsystem is that the user has access to the Firestore database. Because Firestore is not stored locally, that means that an internet connection is required. Another assumption is that the LearnLeaf tools have access to implementations of Firebase's tools and that we are using those tools correctly.

7.2.2 RESPONSIBILITIES

The Learnleaf tools are responsible for:

- performing input validation
- processing user input data into database command entries
- passing queries from pages to the database
- passing query responses from database to pages

7.2.3 SUBSYSTEM INTERFACES

Table 12: LearnLeaf tool Interfaces

ID	Description	Inputs	Outputs
1	Database Update	Entry ID Edit Information	Firebase Edit com- mand with data
2	DB Query	User Input Query	Query Results

8 DATABASE SUBSYSTEMS

The Database layer is the primary source of information for the system overall. It is responsible for maintaining consistency between a user's web and mobile instances, as well as responding to queries or updates from the back-end layers.

8.1 USER

Each user will have an entry in the user list that contains all their information, including lists of all their tasks, subjects, and projects respectively.

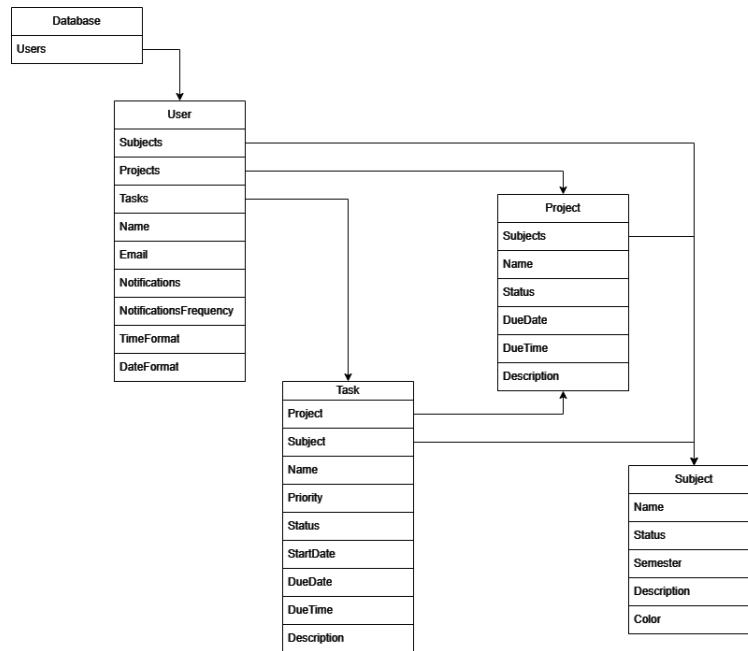


Figure 8: Database Layer Structure

8.1.1 ASSUMPTIONS

A user is not required to have any tasks, subjects, or projects.

8.1.2 RESPONSIBILITIES

The user subsystem manages the user's email information, notification settings, time settings, and date settings. But most importantly it holds lists of all the user's tasks, projects, and subjects.

8.1.3 USER INTERFACES

Each user has a Subjects, Projects, and Tasks item which contains a list of all the tasks subjects, and projects owned by the user.

Table 13: Database User interfaces

ID	Description	Inputs	Outputs
01	Subject List	new or updated subjects	existing subjects
02	Project List	new or updated projects	existing projects
03	Task List	new or updated tasks	existing tasks

8.2 TASKS

Each Task has a name, priority level, completion status, start date, due date, due time, description, as well as a project and subject that it is associated with.

8.2.1 ASSUMPTIONS

A task is not required to have a project or a subject.

8.2.2 RESPONSIBILITIES

The Tasks subsystem is responsible for maintaining all its parameters as well as keeping track of which subject and project it is associated with.

8.2.3 TASK INTERFACES

Table 14: Database Task interfaces

ID	Description	Inputs	Outputs
01	Subject Reference	subject id	subject's data
02	Project Reference	project id	project's data

8.3 PROJECTS

8.3.1 ASSUMPTIONS

A project is not required to have any subjects.

8.3.2 RESPONSIBILITIES

The Tasks subsystem is responsible for:

- **Attribute Management:** Maintains all task-specific information, such as priority and completion status. [1]
- **Associative Tracking:** Tracks each task's association with a subject and project, facilitating organized task grouping. [1]

8.3.3 PROJECT INTERFACES

Table 15: Database Project interfaces

ID	Description	Inputs	Outputs
01	Subject List	new subjects	existing subjects

8.4 SUBJECTS

Each Subject has a name, completion status, semester, color, and description.

8.4.1 ASSUMPTIONS

Subjects can be created without needing to be associated with any project or task.

8.4.2 RESPONSIBILITIES

The Subjects subsystem is responsible for managing all subject-specific information, allowing flexibility in how subjects are associated with tasks or projects. [1]

8.4.3 PROJECT INTERFACES

None. This subsystem acts only as a data structure and does not initiate any interactions.

REFERENCES

- [1] OpenAI. Chatgpt, (2024).